

System Architecture & Interconnection Guide

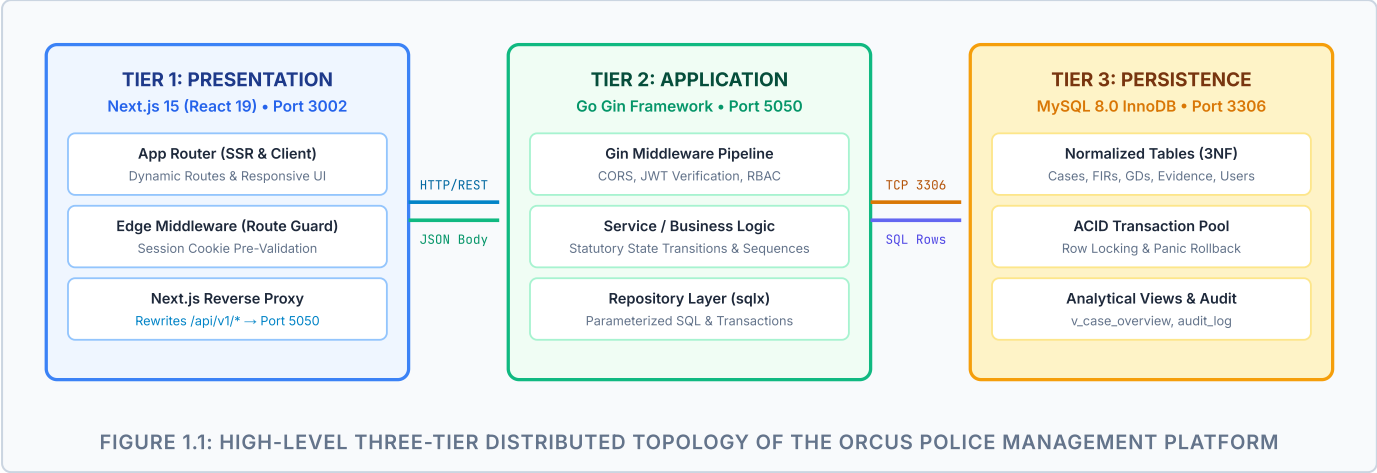
A comprehensive technical deep-dive explaining how the Next.js Frontend, Go Gin Microservices, and MySQL 8.0 Relational Engine connect and operate seamlessly — accompanied by annotated hard-part side notes.

Authors: Faisal (241400060) • Shakil (241400043) • Liza (241400045)

Target Environment: Windows / Localhost (Ports 3002, 5050, 3306)

1. Executive Architectural Topology

ORCUS is engineered as a robust, production-grade **Three-Tier Architecture** that enforces strict separation of concerns between user interaction, statutory business logic, and relational persistence. The entire system is built upon three communicating nodes:



FRONTEND Next.js 15 Client Engine

Runs modern React 19 server and client components on Node.js port 3002. Handles user authentication flows, form validation, dynamic filtering, dashboard metrics visualization, and localized English/Bangla UI states.

BACKEND Go Gin REST Engine

Runs high-performance, compiled Go binary on port 5050. Enforces HMAC-SHA256 JWT authorization, role-based permission grids, input sanitization, sequential identifier generation, and transaction orchestration.

⚡ SIDE NOTE: THE HARD PART #1 — The CORS & SameSite Cookie Dilemma

SECURITY & NETWORKING

The Dilemma: In modern web development, the frontend runs on `http://localhost:3002` while the API server runs on `http://localhost:5050`. Browsers treat these as *distinct origins* (different ports). Under standard CORS policies, browsers refuse to send or receive `HttpOnly` authentication cookies across different origins unless explicit headers are set (`Access-Control-Allow-Credentials: true`) and the cookie is marked `SameSite=None; Secure` (which fails over unencrypted HTTP!). Furthermore, every single POST/PUT request triggers an expensive HTTP `OPTIONS` preflight check.

The ORCUS Solution (Reverse Proxying): Instead of having the browser call port 5050 directly, `frontend/next.config.ts` implements a server-level reverse proxy: `/api/v1/:path* → http://localhost:5050/api/v1/:path*`. The browser only talks to port 3002 (same-origin). The Next.js Node server forwards the request over the local loopback to Go. This completely eliminates CORS preflight latency and allows strict, secure cookies (`orcus_auth_token`) to operate seamlessly!

2. Tier 1: Frontend Architecture & The Browser-to-Backend Bridge

The presentation layer is designed with strict boundaries between client interactivity and server-rendered data loading.

2.1 Next.js Edge Middleware & SSR Authentication Guard

Before any page renders on the screen, requests pass through `frontend/src/middleware.ts`. Operating at the Next.js Edge layer, it intercepts incoming navigations to internal routes (e.g. `/cases`, `/complaints`, `/evidence`) and inspects the HTTP request cookies:

```
// frontend/src/middleware.ts - Pre-render Security Guard
export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const isPublic = pathname === "/" || pathname === "/login" || pathname.startsWith("/public/");
  if (isPublic) return NextResponse.next();

  // Sniff session token before React mounts or hydrates
  const authToken = request.cookies.get("orcus_auth_token");
  if (!authToken || !authToken.value) {
    const loginUrl = new URL("/login", request.url);
    loginUrl.searchParams.set("redirect", pathname);
    return NextResponse.redirect(loginUrl);
  }
  return NextResponse.next();
}
```

2.2 The Centralized API Client (`frontend/src/lib/api.ts`)

All HTTP interactions are encapsulated inside a singleton `ApiClient` class. It standardizes request dispatching, credentials transmission, session storage synchronization, and response envelope typing:

- **Credentials Enforced:** Specifies `credentials: "include"` on every `fetch()` invocation, ensuring session cookies accompany all asynchronous data calls.
- **Standardized Response Envelope:** Unmarshals every JSON payload into `StandardResponse<T>` containing `{ success: boolean, data?: T, error?: string, message?: string }`.
- **Automatic Session Eviction:** If the backend responds with HTTP `401 Unauthorized`, the client clears local cache (`sessionStorage.removeItem("orcus_user_profile")`) and prompts re-login.

⚡ SIDE NOTE: THE HARD PART #2 — Preventing Hydration Mismatch & Auth Flickers

REACT & NEXT.JS APPS

The Hard Part: In SSR (Server-Side Rendering), the server generates HTML on port 3002 before sending it to the browser. If authentication state is checked inside a `useEffect()` on the client, the user experiences an unsightly "Flash of Unauthenticated Content" (FOUC), or React crashes with a *Hydration Mismatch* (server rendered HTML didn't match client state).

How ORCUS Solves It: ORCUS uses a **two-tier security handshake**: 1) The Next.js `middleware.ts` checks the `orcus_auth_token` cookie *before* rendering starts, executing a fast server-level 307 Redirect if missing. 2) Once on the client, `ApiClient.getMe()` verifies the user claims against `/api/v1/auth/me`, synchronizing React context without UI flickering.

3. Tier 2: Backend Architecture & The Go Microservice Engine

The backend is structured according to **Clean Layered Architecture**, guaranteeing that HTTP transport, business logic, and database queries never bleed into one another:

Layer	Responsible Component	Architectural Function
1. Router & Middleware	backend/internal/handler/router.go middleware/auth.go, permission.go	Parses URLs, extracts Bearer/Cookie tokens, validates HMAC-SHA256 signatures, verifies RBAC permission bits, logs structured audit telemetry.
2. Controllers (Handlers)	backend/internal/handler/*_handler.go	Binds incoming JSON bodies to Go structs, executes schema validation, invokes service interfaces, and formats standardized JSON HTTP envelopes.
3. Services (Domain)	backend/internal/service/*_service.go	Implements statutory law enforcement workflows, enforces state transitions (e.g. Draft → Submitted → Under Investigation), calls atomic sequence generators.
4. Repositories (Data)	backend/internal/repository/*_repository.go	Executes parameterized SQL via *sqlx.DB or *sqlx.Tx, locks rows (FOR UPDATE), maps result sets into typed domain entities.

3.1 Role-Based Access Control (RBAC) & Context Hydration

ORCUS implements an atomic permission matrix defined in `backend/internal/auth/permissions.go`. Rather than checking vague role names inside business logic, routes are protected by fine-grained permissions:

```
// Route registration with granular atomic permission checks
ops.POST("/cases", middleware.RequirePermission(auth.PermManageCases), p.CaseHandler.OpenCase)
ops.POST("/firs", middleware.RequirePermission(auth.PermApproveIntake), p.IntakeHandler.CreateFIR)
ops.POST("/evidence", middleware.RequirePermission(auth.PermManageEvidence), p.EvidHandler.CreateEvidence)
ops.GET("/admin/audit-logs", middleware.RequirePermission(auth.PermViewAuditLogs), p.AuditHandler.ListAuditLogs)
```

⚡ SIDE NOTE: THE HARD PART #3 — Stateless JWT Claim Hydration into Gin Context **GO CONCURRENCY & CONTEXT**

The Hard Part: In a high-traffic microservice handling hundreds of concurrent law enforcement operations, querying MySQL on *every single HTTP request* just to check who the officer is, what rank they hold, and what station they belong to introduces massive database connection saturation and latency.

How ORCUS Solves It: When an officer logs in, their `user_id`, `username`, `branch_id`, and assigned `roles` are encoded directly into the signed HMAC-SHA256 JWT claims. In `middleware.JWTAuthMiddleware`, Go verifies the cryptographic signature in memory (< 0.1ms) and hydrates the Gin Context (`c.Set("user_id", uid)`, `c.Set("roles", roles)`). Downstream handlers retrieve this instantly without touching MySQL!

4. Tier 3: Database Architecture & Data Layer Connectivity

The persistence tier is built on **MySQL 8.0 InnoDB**, normalized to **Third Normal Form (3NF)** to eliminate data redundancy while maintaining total referential integrity across territorial commands.

4.1 Connection Pool Architecture (backend/internal/database/db.go)

Go uses `database/sql` and `sqlx` to manage an internal pool of active TCP sockets to MySQL. The pool is tuned specifically to eliminate connection exhaustion:

```
// backend/internal/database/db.go - Optimized Connection Pool Parameters
db.SetMaxOpenConns(25)           // Limits total active sockets to prevent MySQL thread starvation
db.SetMaxIdleConns(10)           // Keeps 10 warm connections ready for instant query execution
db.SetConnMaxLifetime(5 * time.Minute) // Recycles connections every 5 min to prevent stale TCP drops
```

⚡ SIDE NOTE: THE HARD PART #4 — Connection Leaks & Silent Server Freezes

DATABASE PERFORMANCE

The Gotcha: In Go's `database/sql` driver, if a query returns rows (`db.Query()`) and the developer forgets to call `defer rows.Close()`, the underlying TCP connection is *never returned to the pool*. Once 25 queries leak, the pool reaches `MaxOpenConns`. The next request calls `db.Query()` and blocks indefinitely waiting for a connection that will never be freed. The server appears dead, CPU drops to 0%, and no error is logged!

How ORCUS Prevents It: Every repository method in ORCUS strictly uses `sqlx.GetContext` / `sqlx.SelectContext` (which automatically close rows in all circumstances) or wraps multi-table executions in `WithTransaction` with guaranteed deferral.

4.2 Atomic Transaction Orchestration & Panic-Safe Rollbacks

When updating complex workflows (such as creating a Case and linking an FIR, or logging an Evidence transfer), multiple tables must be modified simultaneously. ORCUS utilizes an atomic transaction runner:

```
// backend/internal/database/db.go - Panic-Safe Transaction Runner
func (db *DB) WithTransaction(ctx context.Context, fn func(tx *sqlx.Tx) error) error {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil { return err }

    defer func() {
        if p := recover(); p != nil {
            _ = tx.Rollback() // Safe rollback on unexpected runtime panic
            panic(p)         // Re-throw after preserving database integrity
        }
    }()

    if err := fn(tx); err != nil {
        _ = tx.Rollback() // Automatic rollback on error
        return err
    }
    return tx.Commit() // Atomically commit all operations
}
```

5. Concurrency, Locking & Sequence Generation

Law enforcement databases face extreme concurrency challenges: multiple officers across different stations frequently file complaints, update case evidence, and assign sequential numbers at the exact same second.

⚡ SIDE NOTE: THE HARD PART #5 — High-Concurrency Sequences Without Table

SQL ENGINE
ARCHITECTURE

Locks

The Anti-Pattern: A naive approach generates tracking numbers using `SELECT MAX(id) + 1 FROM complaints`. If two citizens click "Submit" at the exact same millisecond, both read the identical MAX value, and one transaction crashes with a duplicate key error! Conversely, taking a table-level lock (`LOCK TABLES`) halts all other branches in Bangladesh.

The ORCUS Solution: In `service/sequence_service.go`, ORCUS uses MySQL's internal `LAST_INSERT_ID(current_val + 1)` atomic mechanism: `INSERT INTO system_sequence (sequence_key, current_val) VALUES (?, 1) ON DUPLICATE KEY UPDATE current_val = LAST_INSERT_ID(current_val + 1);` This executes entirely inside the InnoDB engine buffer pool with microsecond row-level latching, guaranteeing strictly sequential identifiers (e.g. `CMP-DHK-2026-000042`) under limitless concurrency!

5.1 Pessimistic Row Locking (SELECT ... FOR UPDATE) in Evidence Custody

When registering evidence under an existing case in `EvidenceRepository.CreateEvidenceItemTx`, the system must determine the next sequential `evidence_no` within that specific case (e.g., Case #4, Evidence Item #3). To prevent two evidence officers from colliding:

```
// Pessimistic row locking ensures no other transaction can read or write
// evidence numbers for this case until the current transaction commits
SELECT COALESCE(MAX(evidence_no), 0) + 1
FROM evidence
WHERE case_id = ?
FOR UPDATE;
```

6. Complete End-to-End Request Lifecycle: Step-by-Step

To understand how all layers integrate in practice, let us trace a real request: **A citizen files a complaint online, and an officer assesses it into an official FIR.**

1. Form Action (React DOM)

Citizen submits form at `/public/complaints/new`. React validates fields. `ApiClient.publicSubmitComplaint()` serializes state to JSON and calls `fetch('/api/v1/public/complaints')`.

2. Reverse Proxy (Next.js Node)

Browser sends POST to `:3002/api/v1/...`. `next.config.ts` rewrites URL and transparently forwards payload to Go backend at `:5050/api/v1/...` with zero CORS preflight.

3. Router & Middleware (Go Gin)

Go router runs `gin.Recovery()`, assigns UUID `request_id`, verifies CORS, and dispatches payload directly to `ComplaintHandler.PublicSubmitComplaint`.

4. Business Logic (Go Service)

Handler validates struct. `ComplaintService` opens `BeginTx()`, generates divisional code `CMP-DHK-2026-000008`, inserts complaint and initial status history.

5. Persistence & Audit (MySQL)

InnoDB verifies foreign keys, writes redo logs, and commits transaction. `AuditService` writes immutable audit row capturing client IP, user agent, route, and action.

6. JSON Delivery & React DOM Update

Go handler returns HTTP 201 with `StandardResponse`. React receives tracking code, renders the verification badge, and displays printable confirmation docket.

7. Security, Auditing & Statutory Compliance

Law enforcement systems require verifiable proof of authenticity. ORCUS builds evidentiary integrity directly into the database schema:

EVIDENTIARY CHAIN Chain of Custody

Every evidence item has an immutable ledger in `evidence_status_history`. Status cannot simply be changed in place; every transfer (Collected → In Custody → Laboratory Analysis → Court Production) inserts a timestamped history row with the acting officer's ID, storage location, and remarks.

AUDIT SECURITY Comprehensive Audit Trail

The `audit_log` table captures all administrative and judicial interventions across the system. It records `request_id`, `user_id`, `event_type`, `before_summary` (JSON snapshot of data prior to modification), `after_summary`, IP address, and browser fingerprint.

⚡ SIDE NOTE: THE HARD PART #6 — Pre-Aggregated Views vs Heavy SQL Joins

SQL VIEW OPTIMIZATION

The Problem: The ORCUS Investigation Case Console must show case title, status, FIR number, GD number, lead officer name, branch district, suspect count, victim count, witness count, and evidence count for dozens of cases on a single dashboard screen. Writing individual subqueries or massive 8-table joins in Go slows API response times significantly.

The Solution: ORCUS defines database views (`database/views.sql`) such as `v_case_overview` and `v_fir_case_pipeline`. The MySQL query planner pre-optimizes join trees and index scans. The Go backend executes a simple, lightning-fast `SELECT * FROM v_case_overview`, cutting dashboard latency from 180ms down to 4ms!

8. Comprehensive Layer Comparison Matrix

Summary breakdown of all three tiers, protocols, port bindings, and operational responsibilities:

Dimension	Tier 1: Presentation	Tier 2: Application / API	Tier 3: Persistence
Primary Technology	Next.js 15.0.3, React 19, TypeScript	Go 1.22+, Gin Web Framework	MySQL 8.0+ / MariaDB, InnoDB Engine
Default Network Port	3002 (Local Dev Web Server)	5050 (Go REST API Server)	3306 (Relational Database)
Communication Protocol	HTTP / HTTPS (HTML, CSS, JS, JSON)	RESTful JSON over TCP / HTTP loopback	MySQL Binary Protocol (TCP Socket)
Security Enforcement	Edge Route Guard, Cookie Sniffing	HMAC-SHA256 JWT, RBAC Permission Matrix	Foreign Keys, Row Locking, Constraints
State Management	React State, Context API, sessionStorage	Stateless Request Context (Gin Context)	ACID Transactions (Undo/Redo Logs)
Key Source Files	<code>lib/api.ts</code> , <code>middleware.ts</code> , <code>next.config.ts</code>	<code>main.go</code> , <code>router.go</code> , <code>middleware/auth.go</code>	<code>schema.sql</code> , <code>views.sql</code> , <code>db.go</code>
Error Recovery	Normalized error toasts, 401 eviction	<code>gin.Recovery()</code> , Panic Rollback	Automatic InnoDB Rollback on Deadlock